LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Praktikum -7 Integrasi Backend Server Lokal (Docker & Mosquitto)

dan Rule Engine (Node-RED)

DOSEN PENGAMPU: Dr. Ir. Septafiansyah Dwi Putra S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.



Disusun oleh:

Nama: Cahya Rindi Arouffeti

NPM: 25782035 (3B)

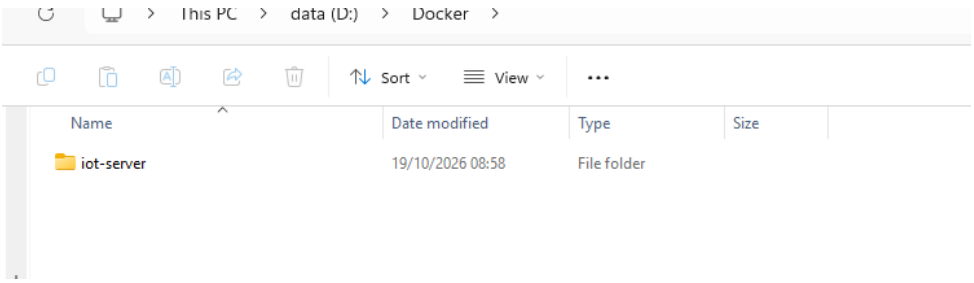
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

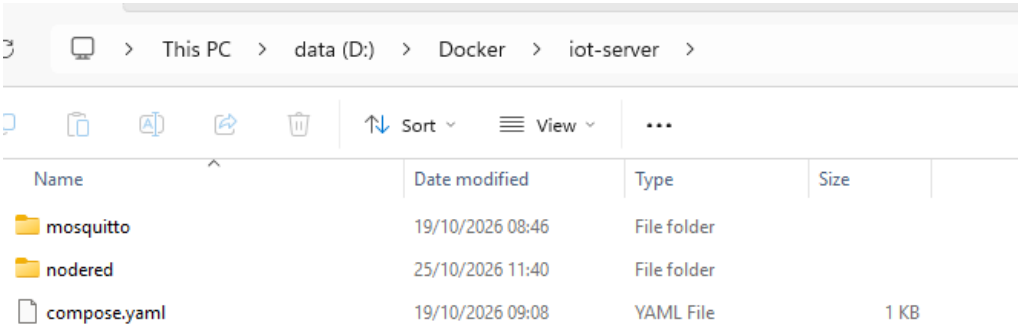
POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

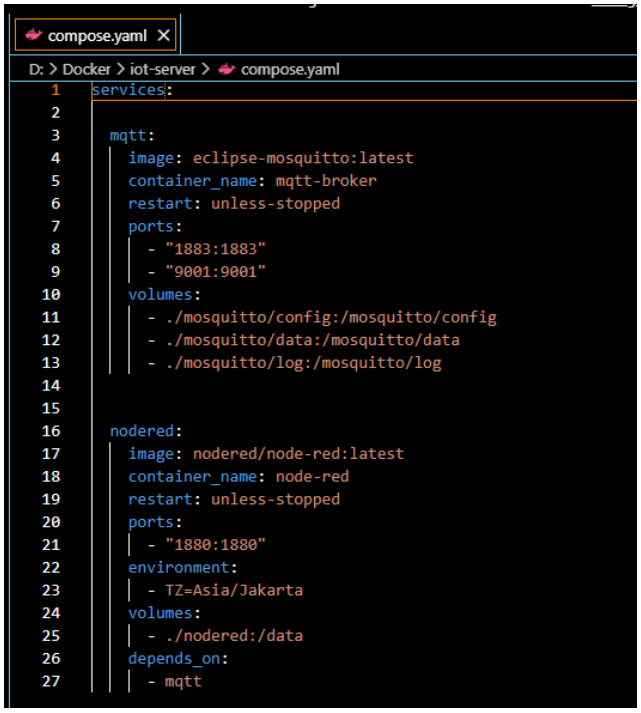
7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose



membuat folder Docker di dalam Data (D) lalu didalam nya buat folder lag Bernama iot-server



Didalam iot-server terdapat 2 folder lagi dan 1 file bernama compose.yaml



Isi didalam file compose.yaml

```
D:\Docker\iot-server>docker compose up -d
[+] up 29/29
 ✔Image nodered/node-red:latest  Pulled
 ✔Image eclipse-mosquitto:latest Pulled
 ✔Network iot-server_default     Created
 ✔Container mqtt-broker          Started
 ✔Container node-red             Started

D:\Docker\iot-server>
```

Eksekusi perintah diatas untuk menjalankan

```
D:\Docker\iot-server>docker compose up -d
[+] up 2/2
 ✔Container node-red    Running
 ✔Container mqtt-broker Running

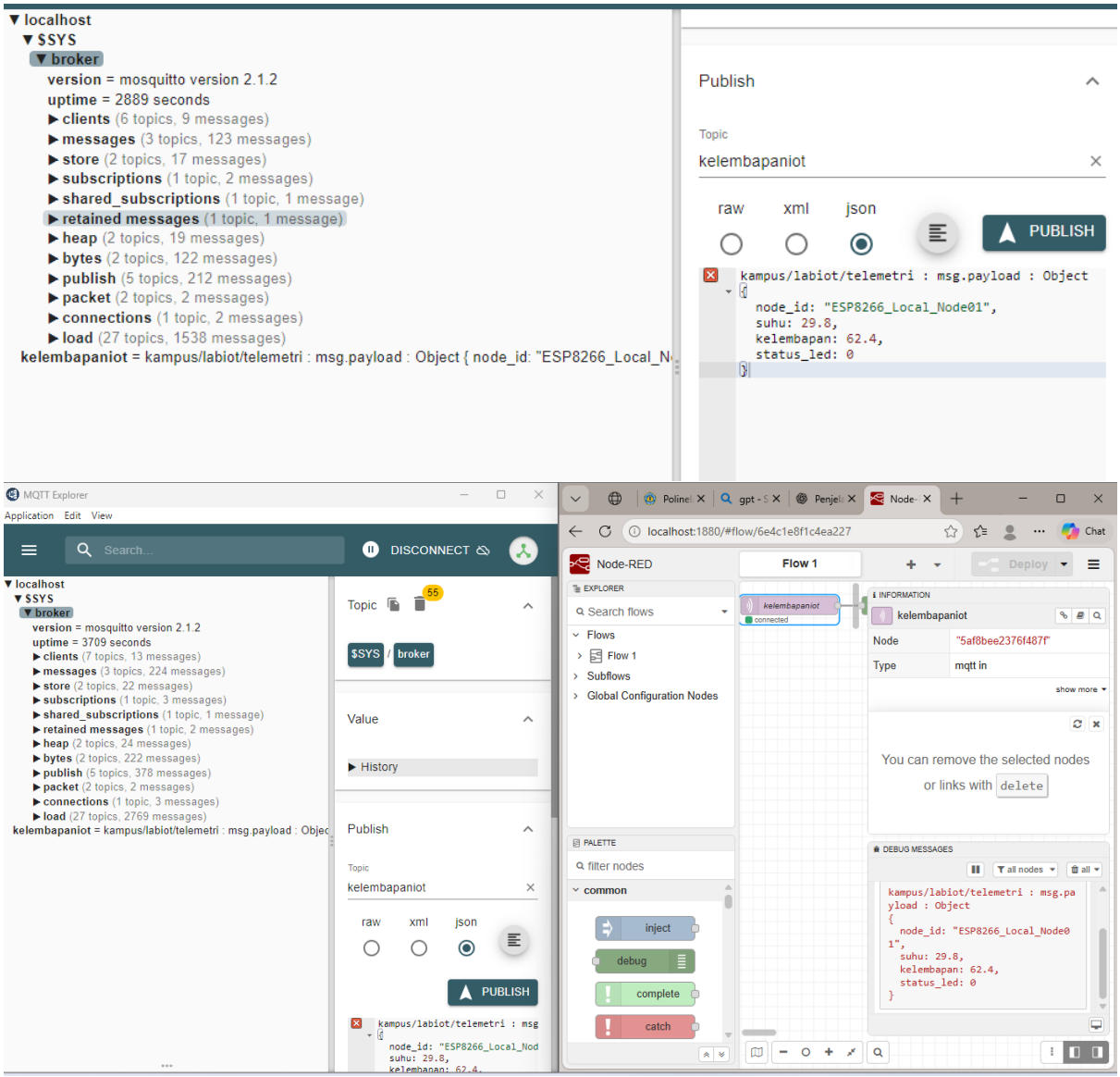
D:\Docker\iot-server>
```

Hasil dari docker compose up -d up 2/2

7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

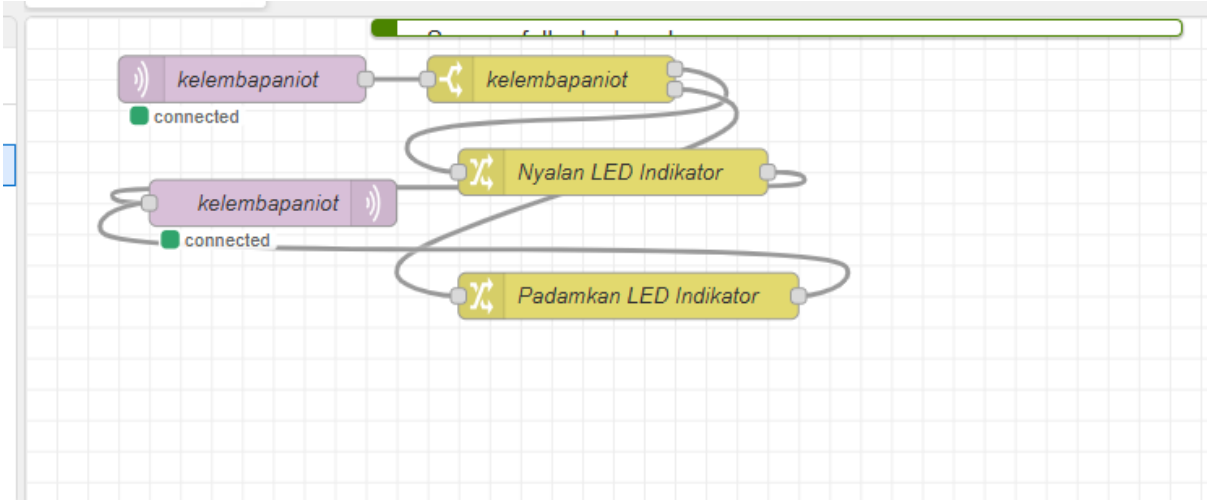
```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.50,"kelembapan":21.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.50,"kelembapan":21.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":1.60,"kelembapan":21.50,"status_led":0}
```

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED



Hasil dari publish mqtt explorer menyambung kan ke node-red menghasilkan keluar seperti gambar diatas

7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup



7.6 Latihan

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):
Penggunaan localhost gagal karena menunjuk ke container Node-RED sendiri, sedangkan nama layanan mqtt dapat digunakan karena Docker Embedded DNS menerjemahkannya ke IP internal broker. Bridge Network memungkinkan kedua container berkomunikasi selama berada pada jaringan yang sama dan broker siap menerima koneksi.

Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):

```
C:\Users\Acer>ping 192.168.56.1

Pinging 192.168.56.1 with 32 bytes of data:
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.56.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Ping menggunakan ip laptop saya sendiri

```
PS C:\Users\Acer> $client = New-Object System.Net.Sockets.TcpClient
PS C:\Users\Acer> $client.Connect("192.168.43.15", 1883)
Exception calling "Connect" with "2" argument(s): "A connection attempt failed because the connected party did not
properly respond after a period of time, or established connection failed because connected host has failed to respond
192.168.43.15:1883"
At line:1 char:1
+ $client.Connect("192.168.43.15", 1883)
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : SocketException

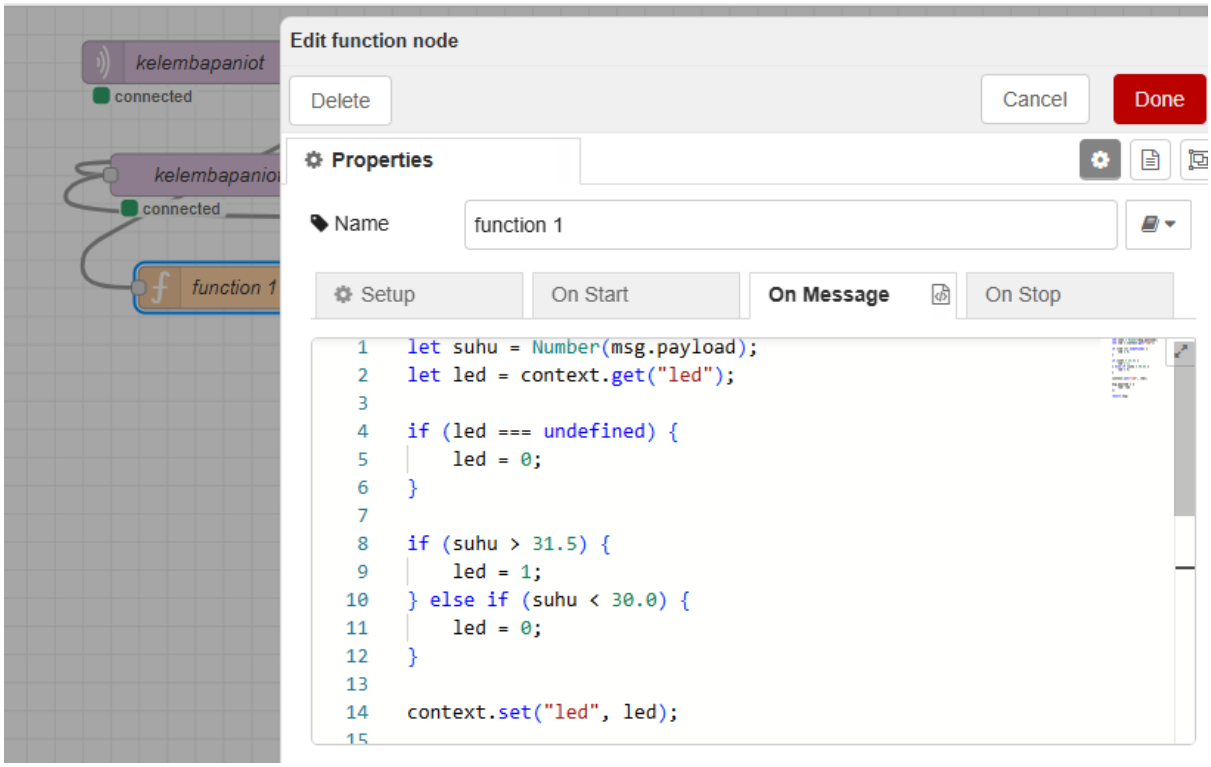
PS C:\Users\Acer> $client.Connected
False
PS C:\Users\Acer> $client.Close()
PS C:\Users\Acer>
```

Menggunakan ip wifi

Karena kedua perangkat berada pada jaringan IP yang berbeda, komunikasi tidak dapat dilakukan secara langsung tanpa perangkat routing yang menghubungkan kedua jaringan tersebut. Jika tidak ada routing yang sesuai, NodeMCU tidak dapat menjangkau broker Mosquitto di laptop untuk memublikasikan data.

Selain itu, koneksi juga dapat terhalang oleh konfigurasi firewall laptop atau pengaturan broker Mosquitto yang tidak mengizinkan koneksi dari perangkat lain.

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):



Menambahkan Function dan memasukan script code menggunakan JavaScript

Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):

```
D:\Docker\iot-server>docker compose config
name: iot-server
services:
  mqtt:
    container_name: mqtt-broker
    image: eclipse-mosquitto:latest
    networks:
      default: null
    ports:
      - mode: ingress
        target: 1883
        published: "1883"
        protocol: tcp
      - mode: ingress
        target: 9001
        published: "9001"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: bind
        source: D:\Docker\iot-server\mosquitto\config
        target: /mosquitto/config
```

Memasukan perintah docker compose config

```
D:\Docker\iot-server>docker compose down
[+] down 3/3
 ✔Container node-red          Removed                    5.6s
 ✔Container mqtt-broker       Removed                    2.2s
 ✔Network iot-server_default  Removed                    0.5s
```

Kontainer dimatikan menggunakan perintah docker compose down

```
D:\Docker\iot-server>docker compose up -d
[+] up 3/3
 ✔Network iot-server_default  Created
 ✔Container mqtt-broker       Started
 ✔Container node-red          Started

D:\Docker\iot-server>
```

Menghidupkan Kembali menggunakan perintah docker compose up -d

Bind Mount digunakan agar data tetap tersimpan di laptop meskipun container dihapus. Named Volume juga bisa digunakan untuk mempertahankan data, tetapi penyimpanannya dikelola oleh Docker.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
PS C:\Users\Acer> docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/passw
ord.txt admin
Password:
Reenter password:
Adding password for user admin
```

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous false

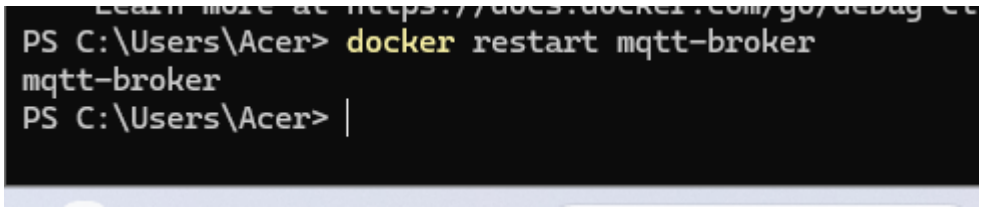
password_file /mosquitto/config/password.txt

persistence true
persistence_location /mosquitto/data/

log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

Mengubah allow_anonymous menjadi false dan menambahkan teks baru dengan “password_file /mosquito/config/password.txt”

3. Memuat Ulang Layanan Broker:



Restart kontainer `mqtt-broker` melalui terminal agar kebijakan keamanan baru diterapkan

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

```
Wi-Fi] Berhasil Terhubung!
Wi-Fi] IP NodeMCU: 172.20.10.9
MQTT] Menghubungkan ke Broker Lokal (192.168.43.48)... Gagal (rc=-2). Coba lagi dalam 4 detik...
MQTT] Menghubungkan ke Broker Lokal (192.168.43.48)... Gagal (rc=-2). Coba lagi dalam 4 detik...
MQTT] Menghubungkan ke Broker Lokal (192.168.43.48)... Gagal (rc=-2). Coba lagi dalam 4 detik...
```

5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

```
void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
    Serial.print(mqttServer);
    Serial.print(")...");

    // Format: client.connect(clientId, username, password)
    if (client.connect(clientId, "admin", "rahasia123")) {
      Serial.println(" Terhubung dengan Autentikasi Sah!");

      // Segera daftarkan subskripsi topik kendali
      client.subscribe(topicSubCmd);
      Serial.printf(
        "[MQTT] Berlangganan ke topik: %s\n",
        topicSubCmd
      );
    } else {
      Serial.printf(
        " Gagal (rc=%d). Coba lagi dalam 4 detik...\n",
        client.state()
      );
      delay(4000);
    }
  }
}
```

Pemutakhiran Konfigurasi Kredensial pada Node-RED

